PEMROGRAMAN MOBILE // MATERI KULIAH TEKNIK INFORMATIKA

Android Sensor Framework

Arsitektur, Siklus Hidup, dan Implementasi Sensor Native Menggunakan Java di Android Studio

PLATFORM
Android Studio

BAHASA
Java Native (Java 8 / 11 / 17)

TARGET SDK
Android API 34+ (Android 14)

FOKUS
Motion, Position & Environment

TUJUAN PEMBELAJARAN

Mahasiswa Menguasai 4 Pilar Pemrograman Sensor Android Berbasis Java

1. Arsitektur Sensor Stack

Memahami aliran data dari lapisan driver hardware Linux, HAL (Hardware Abstraction Layer), Android Framework C++, hingga Java Application Layer.

2. Pemrograman API Java

Mampu menginisialisasi SensorManager, mengidentifikasi objek Sensor, serta mengimplementasikan callback SensorEventListener di Java.

3. Manajemen Daya & Siklus Hidup

Menguasai registrasi listener pada onResume() dan pembatalan pada onPause(), serta pemilihan sampling rate yang hemat baterai.

4. Studi Kasus Industri Riil

Membangun fitur interaktif: Shake Detection, Deteksi Jarak Proximity (Screen-Off), Auto Dark Mode Lux, dan Kompas Digital.

KONSEP DASAR

Sensor Mengubah Ponsel Pasif Menjadi Perangkat Kontekstual yang Cerdas

Context-Aware Computing

Ponsel memahami situasi fisik pengguna di dunia nyata tanpa masukan manual.

- Mendeteksi apakah pengguna sedang berjalan, berlari, atau berkendara.
- Mengetahui orientasi fisik perangkat (tegak, mendatar, miring).
- Mendeteksi tingkat pencahayaan ruangan sekitar secara real-time.

Natural User Interface (NUI)

Interaksi aplikasi yang intuitif menggantikan tombol visual konvensional.

- Menggoyangkan ponsel (shake) untuk membatalkan input (Undo) atau mengacak data.
- Membalik ponsel menghadap ke bawah (flip-to-mute) untuk mendiamkan alarm.
- Menyentuh atau mendekatkan ponsel ke telinga untuk menerima panggilan.

Otomasi & Efisiensi Sistem

Sistem operasi mengambil tindakan proteksi hardware secara cerdas.

- Mematikan layar sentuh saat didekatkan ke wajah untuk mencegah salah sentuh.
- Menyesuaikan backlight layar secara dinamis guna memperpanjang daya baterai.
- Membantu stabilisasi gambar optik (OIS) kamera saat merekam video.

KLASIFIKASI SENSOR

Android Mengelompokkan Sensor ke dalam Tipe Fisik (MEMS) dan Sensor Sintetis

HARDWARE SENSORS (MEMS FISIK)

Komponen silikon fisik terpasang langsung di motherboard perangkat yang mengukur besaran fisika murni:

- TYPE_ACCELEROMETER: Mengukur percepatan (m/s^2).
- TYPE_GYROSCOPE: Mengukur kecepatan rotasi sudut (rad/s).
- TYPE_MAGNETIC_FIELD: Mengukur medan geomagnetik (µT).
- TYPE_LIGHT: Mengukur intensitas cahaya sekitar (lux).
- TYPE_PROXIMITY: Mengukur jarak objek ke layar (cm).
- TYPE_PRESSURE: Mengukur tekanan udara atmosfer (hPa).

SYNTHETIC / SOFTWARE SENSORS

Sensor virtual yang tidak memiliki hardware independen; datanya dihitung dari fusi beberapa sensor fisik oleh sistem Android:

- TYPE_GRAVITY: Memisahkan komponen murni gravitasi dari percepatan total.
- TYPE_LINEAR_ACCELERATION: Murni gaya percepatan pengguna tanpa gravitasi bumi.
- TYPE_ROTATION_VECTOR: Menghitung orientasi 3D 4-elemen kuaternion (fusi accel + gyro).
- TYPE_STEP_DETECTOR: Memicu event saat satu langkah terdeteksi.
- TYPE_STEP_COUNTER: Akumulasi total langkah pengguna sejak perangkat aktif.

SISTEM KOORDINAT

Android Menetapkan Standar Koordinat Kartesius 3 Sumbu yang Terikat pada Perangkat

Sumbu X (Horizontal)

Arah mendatar dari kiri ke kanan perangkat.

- Nilai Positif (+): Perangkat bergerak ke kanan.
- Nilai Negatif (-): Perangkat bergerak ke kiri.
- Memiringkan ponsel ke kiri menyebabkan gravitasi menarik ke sumbu X negatif.

Sumbu Y (Vertikal)

Arah tegak dari bawah ke atas perangkat.

- Nilai Positif (+): Perangkat bergerak ke atas.
- Nilai Negatif (-): Perangkat bergerak ke bawah.
- Saat ponsel berdiri tegak (portrait), gravitasi bumi (9.81 m/s^2) bekerja pada sumbu Y positif.

Sumbu Z (Kedalaman)

Arah tegak lurus menembus layar ponsel.

- Nilai Positif (+): Mengarah ke depan (menuju wajah pengguna).
- Nilai Negatif (-): Mengarah ke belakang bodi ponsel.
- Saat diletakkan terlentang di atas meja, sumbu Z positif menahan percepatan gravitasi bumi.

PENTING: KOORDINAT TERIKAT PADA PERANGKAT (DEVICE-FRAME)

Sumbu koordinat sensor TIDAK berubah saat pengguna memutar layar dari Portrait ke Landscape. Sumbu X tetap merupakan sisi pendek ponsel, dan sumbu Y adalah sisi panjang ponsel.

BAGIAN II // ARSITEKTUR & KOMPONEN INTI

Arsitektur Sensor Stack & Komponen Inti Java

Mendalami 4 Lapisan Sensor Android, SensorManager, Sensor, SensorEvent, dan SensorEventListener di Java

ARSITEKTUR SISTEM

Data Sensor Mengalir Melalui 4 Lapisan dari Kernel Linux hingga Aplikasi Java

1. Application Layer (Java)

Aplikasi Android pengembang yang mengimplementasikan SensorEventListener untuk menerima objek SensorEvent dan memperbarui tampilan UI.

2. Android Framework (Java & JNI)

Menyediakan API android.hardware.SensorManager, menangani pendaftaran multi-listener, filter sampling rate, dan komunikasi IPC Binder.

3. Native & Hardware Abstraction Layer (HAL)

Modul C/C++ (libsensors.so / sensors.default.so) yang menyediakan antarmuka standar bagi sistem Android untuk berkomunikasi dengan driver vendor.

4. Linux Kernel Drivers

Driver perangkat keras tingkat rendah (driver I2C / SPI) yang berkomunikasi langsung dengan sensor MEMS fisik pada papan sirkuit ponsel.

KOMPONEN INTI JAVA

SensorManager Mengatur Layanan Sensor dan Objek Sensor Menyimpan Representasi Hardware

1. SENSORMANAGER

Layanan sistem (System Service) sentral yang mengontrol seluruh operasi sensor di Android.

Cara Inisialisasi di Java:
SensorManager sm = (SensorManager)
getSystemService(Context.SENSOR_SERVICE);

Metode Vital:
- getDefaultSensor(int type): Mengambil instance sensor default.
- getSensorList(int type): Mengembalikan daftar seluruh sensor.
- registerListener(): Mendaftarkan callback penangkap event.
- unregisterListener(): Melepas callback untuk menghemat daya.

2. SENSOR (KELAS REPRESENTASI)

Instance kelas yang merepresentasikan satu hardware atau software sensor spesifik pada perangkat.

Cara Mengambil di Java:
Sensor accelerometer =
sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);

Pemeriksaan Ketersediaan (Null-Check):
```
if (accelerometer == null) {
   // Perangkat tidak memiliki sensor ini
}
```

Metadata Hardware yang Dapat Dibaca:
- getName(): Merk dagang sensor.
- getVendor(): Pabrikan pembuat (Bosch, Asahi, dll).
- getPower(): Arus listrik yang dikonsumsi (mA).
- getResolution(): Ketelitian terkecil pembacaan.

CALLBACK PATTERN

SensorEventListener Menjadi Jembatan Callback untuk Menangkap Objek SensorEvent

INTERFACE SENSOREVENTLISTENER

Diimplementasikan oleh Activity atau Helper Class di Java:

1. public void onSensorChanged(SensorEvent event)
Dipanggil setiap kali perangkat mendeteksi perubahan data baru. Berjalan pada thread UI secara default.

2. public void onAccuracyChanged(Sensor sensor, int accuracy)
Dipanggil jika akurasi kalibrasi hardware berubah (contoh: kompas perlu dikalibrasi angka 8).

Level Akurasi Standar Android:
- SENSOR_STATUS_ACCURACY_HIGH
- SENSOR_STATUS_ACCURACY_MEDIUM
- SENSOR_STATUS_ACCURACY_LOW
- SENSOR_STATUS_UNRELIABLE

STRUKTUR DATA SENSOREVENT

Objek yang dikirimkan oleh sistem ke dalam method onSensorChanged():

- event.sensor: Referensi ke objek Sensor asal data.
- event.timestamp: Waktu kejadian dalam nanodetik (uptime sistem).
- event.accuracy: Tingkat akurasi saat data dibaca.
- event.values (float[]): Array pembacaan data sensor:
  - event.values[0] = Nilai Sumbu X
  - event.values[1] = Nilai Sumbu Y
  - event.values[2] = Nilai Sumbu Z

PERINGATAN PERFORMA: Objek SensorEvent didaur ulang oleh Android. Jangan simpan referensi objek event secara langsung!

IMPLEMENTASI JAVA

Mengidentifikasi Seluruh Sensor yang Tersedia Menggunakan getSensorList() di Java

SensorListInspectorActivity.java

```java
// SensorListInspectorActivity.java
package com.example.sensordemo;

import android.content.Context;
import android.hardware.Sensor;
import android.hardware.SensorManager;
import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;
import java.util.List;
public class SensorListInspectorActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_sensor_list);

        SensorManager sm = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
        List<Sensor> sensorList = sm.getSensorList(Sensor.TYPE_ALL);

        Log.d("SENSOR_COUNT", "Total Sensor Terpasang: " + sensorList.size());
        for (Sensor sensor : sensorList) {
            Log.d("SENSOR_INFO", String.format(
                "Nama: %s | Vendor: %s | Daya: %.2f mA | Resolusi: %.4f",
                sensor.getName(), sensor.getVendor(),
                sensor.getPower(), sensor.getResolution()
            ));
        }
    }
}
```

PENJELASAN KODE JAVA

- Sensor.TYPE_ALL digunakan untuk meminta seluruh sensor, baik fisik maupun sintetis.
- getSensorList() mengembalikan objek java.util.List<Sensor> yang dapat diiterasi dengan for-each loop.
- sensor.getPower() mengembalikan konsumsi daya dalam miliampere (mA), krusial untuk audit efisiensi.
- sensor.getResolution() menunjukkan batas sensitivitas terkecil sensor terhadap perubahan fisik.

BAGIAN III // SIKLUS HIDUP & EFISIENSI DAYA

Siklus Hidup (Lifecycle) & Efisiensi Energi

Manajemen Delay Sampling Rate, Pola Baku onResume & onPause, serta Fallback Hardware di Java

EFISIENSI DAYA

Pemilihan Sensor Delay Rate Menentukan Keseimbangan Antara Akurasi dan Baterai

Konstanta Delay	Interval Waktu	Frekuensi Sampling	Rekomendasi Penggunaan
SENSOR_DELAY_NORMAL	~200.000 µs (200 ms)	5 Hz (5 kali / detik)	Perubahan orientasi layar dasar & penghematan daya baterai maksimal.
SENSOR_DELAY_UI	~60.000 µs (60 ms)	16.6 Hz (~17 kali / detik)	Pembaruan antarmuka grafis standar yang nyaman di mata manusia.
SENSOR_DELAY_GAME	~20.000 µs (20 ms)	50 Hz (50 kali / detik)	Kontrol kemiringan game mobil balap dan interaksi gestur real-time.
SENSOR_DELAY_FASTEST	0 µs (Maksimum)	100 - 400+ Hz (Sesuai HW)	Hanya untuk kalibrasi instrumen laboratorium (Sangat boros baterai).

PRINSIP EMAS EFISIENSI: GUNAKAN SAMPLING RATE TERENDAH YANG MEMADAI

Android Framework memperlakukan delay rate sebagai petunjuk (hint). Sistem dapat memilih sampling rate lebih tinggi jika aplikasi lain meminta rate yang lebih cepat secara simultan.

LIFECYCLE MANAGEMENT

Sensor Wajib Didaftarkan di onResume() dan Dilepas di onPause() untuk Mencegah Kebocoran

onCreate()

Inisialisasi Objek

Inisialisasi SensorManager dan ambil referensi Sensor. JANGAN daftarkan listener di sini!

onResume()

Mulai Mendengarkan

Panggil registerListener(). Sensor hardware mulai aktif mengirimkan event ke aplikasi.

onPause()

Hentikan Sensor

WAJIB panggil unregisterListener(). Sistem Android mematikan chip sensor untuk hemat baterai.

onDestroy()

Pembersihan Memori

Lepaskan seluruh referensi objek untuk mencegah memory leak pada Garbage Collector.

PERINGATAN: WAJIB MEMANGGIL UNREGISTERLISTENER() DI ONPAUSE()

Jika listener tidak dilepas, chip sensor dan CPU akan terus berjalan di latar belakang (background) meskipun layar mati, menghabiskan baterai dalam hitungan jam dan menyebabkan aplikasi ditandai boros daya oleh sistem Android.

KODE BAKU JAVA

Implementasi Pola Standar Pendaftaran dan Pelepasan Listener di Android Java

MainActivity.java (Pola Standar)

```
public class MainActivity extends AppCompatActivity implements SensorEventListener {
    private SensorManager sensorManager;
    private Sensor accelerometer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        sensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
        if (sensorManager != null) {
            accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
        }
    }
 @Override
    protected void onResume() {
        super.onResume();
        // Daftarkan listener saat Activity kembali ke layar depan (Foreground)
        if (accelerometer != null) {
            sensorManager.registerListener(this, accelerometer, SensorManager.SENSOR_DELAY_UI);
        }
    }
@Override
    protected void onPause() {
        super.onPause();
        // Wajib batalkan pendaftaran saat Activity tidak aktif (Background)
        if (sensorManager != null) {
            sensorManager.unregisterListener(this);
        }
    }
}
```

POIN PENTING JAVA

1. null-check accelerometer:
Mencegah NullPointerException jika perangkat tidak memiliki sensor.

2. registerListener(this, ...):
Parameter 'this' menandakan Activity saat ini yang mengimplementasikan SensorEventListener.

3. unregisterListener(this):
Cukup panggil satu baris ini untuk melepas seluruh sensor yang didengarkan oleh Activity.

KOMPATIBILITAS

Deklarasikan Fitur Bersifat Opsional dan Hadirkan Graceful Fallback di Java

KonfigurasiFiturSensor.txt

```
// Konfigurasi Fitur Hardware pada Proyek:

// 1. Deklarasi Fitur Opsional (required = false):
Fitur: android.hardware.sensor.accelerometer
Status: required = false (Aplikasi tetap dapat diinstal)

Fitur: android.hardware.sensor.proximity
Status: required = false (Opsional)

Fitur: android.hardware.sensor.light
Status: required = false (Opsional)

// Alasan Penting:
Menghindari pemblokiran distribusi aplikasi di Google Play Store
bagi tablet atau perangkat khusus tanpa sensor tertentu.
```

GracefulFallback.java

```
// Pemeriksaan Ketersediaan di Java (Graceful Fallback)
PackageManager pm = getPackageManager();
boolean hasAccel = pm.hasSystemFeature(
    PackageManager.FEATURE_SENSOR_ACCELEROMETER
);

if (accelerometer == null || !hasAccel) {
    // Sensor fisik tidak ditemukan pada perangkat
    tvSensorStatus.setText("Sensor Tidak Didukung");
    btnShakeFeature.setEnabled(false);
    btnShakeFeature.setAlpha(0.5f);

    Toast.makeText(this,
        "Fitur dinonaktifkan: Perangkat tanpa Accelerometer",
        Toast.LENGTH_LONG
    ).show();
}
```

BAGIAN IV // SENSOR GERAK (MOTION SENSORS)

Sensor Gerak (Motion Sensors) & Praktik Java

Fisika Accelerometer, Gyroscope, Algoritma Shake Detection, dan Sensor Fusion di Java

MOTION SENSORS

Accelerometer Mengukur Gaya Percepatan Bersih Termasuk Efek Gravitasi (9.81 m/s²)

Kondisi 1: Diam Terlentang di Meja

- Sumbu X = 0.0 m/s²
- Sumbu Y = 0.0 m/s²
- Sumbu Z = +9.81 m/s² (GRAVITY_EARTH)
- Penjelasan: Meja memberikan gaya normal ke atas menahan gravitasi bumi.

Kondisi 2: Berdiri Tegak (Portrait)

- Sumbu X = 0.0 m/s²
- Sumbu Y = +9.81 m/s²
- Sumbu Z = 0.0 m/s²
- Penjelasan: Gravitasi bumi menarik ke arah bawah bodi ponsel (sepanjang sumbu Y).

Kondisi 3: Gerak Melayang di Udara

- Sumbu X = 0.0 m/s²
- Sumbu Y = 0.0 m/s²
- Sumbu Z = 0.0 m/s²
- Penjelasan: Saat melayang tanpa tahanan, percepatan bersih bernilai nol.

ALTERNATIF: MURNI PERCEPATAN PENGGUNA TANPA GRAVITASI

Gunakan sensor sintetis Sensor.TYPE_LINEAR_ACCELERATION. Sistem Android akan otomatis memfilter komponen gravitasi 9.81 m/s², sehingga saat diam nilainya murni 0.0 m/s².

PANDUAN LANGKAH

Tiga Tahap Terstruktur untuk Membaca Data Accelerometer ke Antarmuka Pengguna

Langkah 1: Deklarasi & Binding

- Buat Activity baru di Android Studio.
- Deklarasikan variabel SensorManager, Sensor, dan TextView UI di MainActivity.java.
- Inisialisasi SensorManager via getSystemService().
- Ambil instance Sensor.TYPE_ACCELEROMETER dan pastikan tidak bernilai null.

Langkah 2: Registrasi Listener

- Tambahkan implements SensorEventListener pada deklarasi class.
- Override method onResume() dan panggil registerListener().
- Pilih sampling rate SENSOR_DELAY_UI.
- Override method onPause() dan panggil unregisterListener().

Langkah 3: Ekstraksi Data Sumbu

- Override method onSensorChanged(SensorEvent event).
- Verifikasi event.sensor.getType() == Sensor.TYPE_ACCELEROMETER.
- Ekstrak nilai float x = event.values[0], y = event.values[1], z = event.values[2].
- Format angka desimal dan tampilkan ke TextView.

IMPLEMENTASI JAVA

Kode Lengkap Membaca Nilai Sumbu Accelerometer Secara Real-Time ke Tampilan UI

AccelerometerActivity.java

```
public class AccelerometerActivity extends AppCompatActivity implements SensorEventListener {
    private SensorManager sensorManager;
    private Sensor accelerometer;
    private TextView tvValues;
 @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_accelerometer);
        tvValues = findViewById(R.id.tvAccelerometerValues);

        sensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
    }
@Override
    public void onSensorChanged(SensorEvent event) {
        if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
            float x = event.values[0];
            float y = event.values[1];
            float z = event.values[2];

            String data = String.format(Locale.getDefault(),
                "X: %.2f m/s²\nY: %.2f m/s²\nZ: %.2f m/s²", x, y, z);
            tvValues.setText(data);
        }
    }
@Override
    public void onAccuracyChanged(Sensor sensor, int accuracy) {}
}
```

ANALISIS RUNTIME

1. Thread UI Aman:
Callback onSensorChanged() secara default berjalan di Main UI Thread, sehingga pemanggilan tvValues.setText() langsung aman dilakukan.

2. String.format():
Mencegah teks bergetar dengan membatasi format desimal ke 2 angka di belakang koma (%.2f).

3. Efisiensi Alokasi:
Hindari membuat StringBuilder atau objek kompleks baru di dalam onSensorChanged().

STUDI KASUS 1

Mendeteksi Guncangan Pengguna Menggunakan Kalkulasi Magnitude & Time Debounce

ShakeDetectorActivity.java

```
public class ShakeDetectorActivity extends AppCompatActivity implements SensorEventListener {
    private static final float SHAKE_THRESHOLD = 12.0f; // m/s² di atas gravitasi
    private long lastShakeTimestamp = 0;

    @Override
    public void onSensorChanged(SensorEvent event) {
        if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
            float x = event.values[0];
            float y = event.values[1];
            float z = event.values[2];

            // Hitung total magnitude vektor percepatan 3 dimensi
            double magnitude = Math.sqrt(x * x + y * y + z * z);
            double delta = Math.abs(magnitude - SensorManager.GRAVITY_EARTH);
            long now = System.currentTimeMillis();

            // Evaluasi ambang batas dan debounce time 1000 ms
            if (delta > SHAKE_THRESHOLD && (now - lastShakeTimestamp > 1000)) {
                lastShakeTimestamp = now;
                onDeviceShaken();
            }
        }
    }

    private void onDeviceShaken() {
        Toast.makeText(this, "Guncangan Terdeteksi! Refreshing...", Toast.LENGTH_SHORT).show();
    }
}
```

LOGIKA MATEMATIKA

1. Teorema Pythagoras 3D:
Magnitude = $\sqrt{(x^2 + y^2 + z^2)}$.
Menghitung total energi percepatan ke segala arah.

2. Kompensasi Gravitasi:
Delta = |Magnitude - 9.81|.
Mengeliminasi tarikan konstan gravitasi bumi.

3. Debounce Timer (1000 ms):
Mencegah satu kali sentakan fisik memicu callback puluhan kali berturut-turut.

MOTION SENSORS

Gyroscope Mengukur Kecepatan Sudut Rotasi Sumbu dalam Radian per Detik (rad/s)

Sumbu X (Pitch)

Rotasi memiringkan ponsel ke depan atau belakang.

- Memutar ponsel ke arah wajah: Kecepatan sudut positif.
- Memutar menjauh dari wajah: Kecepatan sudut negatif.
- Satuan pengukuran: Radian/detik (rad/s).

Sumbu Y (Roll)

Rotasi memiringkan ponsel ke sisi samping.

- Memiringkan ponsel ke kiri atau kanan.
- Digunakan sebagai kendali kemudi setir pada game mobil balap.
- Sangat sensitif terhadap putaran pergelangan tangan.

Sumbu Z (Yaw)

Rotasi memutar ponsel di atas meja.

- Memutar ponsel searah atau berlawanan jarum jam saat ponsel diletakkan datar.
- Membantu penentuan arah belokan pada peta navigasi indoor.

INISIALISASI GYROSCOPE DI JAVA

```
Sensor gyro = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);
Nilai: event.values[0] = rad/s sumbu X, event.values[1] = rad/s sumbu Y, event.values[2] = rad/s sumbu Z.
```

SENSOR FUSION

Sensor Fusion Mengatasi Kelemahan Noise Accelerometer dan Hanyutan (Drift) Gyroscope

Karakteristik Accelerometer

Kelebihan:
• Akurat untuk jangka panjang karena memiliki referensi gravitasi bumi konstan.

Kelemahan:
• Sangat sensitif terhadap getaran sesaat (High-Frequency Noise).

Karakteristik Gyroscope

Kelebihan:
• Sangat responsif, halus, dan bebas dari noise getaran mekanis sesaat.

Kelemahan:
• Integrasi kecepatan ke sudut mengalami akumulasi kesalahan (Gyro Drift) jangka panjang.

Solusi Fusi (Complementary Filter)

Kombinasi Matematika:
Sudut = 0.98 * (Sudut + Gyro * dt) + 0.02 * (Accel)

• Mengambil respon cepat Gyroscope.
• Dikoreksi kestabilan jangka panjang Accelerometer.

BAGIAN V // SENSOR POSISI & LINGKUNGAN

Sensor Posisi & Lingkungan (Position & Environment)

Proximity Sensor, Ambient Light Sensor, dan Digital Compass di Java

POSITION SENSOR

Proximity Sensor Menggunakan Refleksi Inframerah dan Mengembalikan Nilai Biner

Mekanisme Inframerah (IR)

Memancarkan sinar inframerah tak kasat mata di samping kamera depan. Jika ada objek (seperti telinga pengguna) mendekat, sinar dipantulkan kembali ke fotodioda sensor.

Karakteristik Nilai Hardware Biner

Meskipun bertipe float dalam satuan sentimeter, mayoritas produsen hardware hanya mengembalikan 2 nilai diskrit:

- 0.0 cm (Kondisi NEAR / Sangat Dekat)
- maximumRange (Kondisi FAR / Jauh, misal 5.0 cm)

Aturan Pemrograman Aman di Java

JANGAN PERNAH mengecek: if (event.values[0] == 0.0f)!

Selalu bandingkan terhadap maximumRange:

```
if (event.values[0] < proximitySensor.getMaximumRange()) {
// Objek sedang dekat
}
```

STUDI KASUS 2

Simulasi Mematikan Layar Panggilan Telepon Berdasarkan Jarak Objek di Java

ProximityActivity.java

```
public class ProximityActivity extends AppCompatActivity implements SensorEventListener {
    private SensorManager sensorManager;
    private Sensor proximitySensor;
    private View screenOverlay;
    private TextView tvStatus;
 @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_proximity);
        screenOverlay = findViewById(R.id.screenOverlay);
        tvStatus = findViewById(R.id.tvProximityStatus);
       sensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
        proximitySensor = sensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY);
    }
 @Override
    public void onSensorChanged(SensorEvent event) {
        if (event.sensor.getType() == Sensor.TYPE_PROXIMITY) {
            float distance = event.values[0];
            float maxRange = proximitySensor.getMaximumRange();

            if (distance < maxRange) {
                // Objek dekat telinga: Matikan tampilan layar (Layar Hitam)
                screenOverlay.setVisibility(View.VISIBLE);
                tvStatus.setText("Status: DEKAT (Layar Dinonaktifkan)");
            } else {
                // Objek menjauh: Tampilkan kembali antarmuka
                screenOverlay.setVisibility(View.GONE);
                tvStatus.setText("Status: JAUH (Layar Aktif)");
            }
        }
    }
    @Override public void onAccuracyChanged(Sensor sensor, int accuracy) {}
}
```

MANFAAT FITUR

1. Proteksi Layar:
Mencegah pipi atau telinga menekan tombol dialer secara tidak sengaja.

2. Hemat Baterai:
Mematikan backlight display selama percakapan telepon berlangsung.

3. Uji Coba Cepat:
Cukup letakkan telapak tangan di atas bagian atas ponsel untuk menguji.

STUDI KASUS 3

Mengukur Intensitas Cahaya (Lux) untuk Memicu Peralihan Mode Gelap Otomatis di Java

LightSensorActivity.java

```
public class LightSensorActivity extends AppCompatActivity implements SensorEventListener {
    private SensorManager sensorManager;
    private Sensor lightSensor;
    private TextView tvLux;

    @Override
    public void onSensorChanged(SensorEvent event) {
        if (event.sensor.getType() == Sensor.TYPE_LIGHT) {
            float lux = event.values[0];
            tvLux.setText(String.format(Locale.getDefault(), "Cahaya: %.1f Lux", lux));

            // Hysteresis threshold untuk mencegah flicker tema
            if (lux < 10.0f) {
                // Ruangan gelap: Aktifkan Dark Mode
                AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_YES);
            } else if (lux > 80.0f) {
                // Ruangan terang: Kembalikan ke Light Mode
                AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO);
            }
        }
    }
    @Override public void onAccuracyChanged(Sensor sensor, int accuracy) {}
}
```

SKALA ILUMINASI LUX

- 0 - 10 Lux:
  Malam hari / Bioskop / Ruang gelap total.
- 10 - 50 Lux:
  Pencahayaan lilin / Lorong redup.
- 100 - 500 Lux:
  Ruang kerja kantor & ruang kuliah standar.
- 1.000 - 10.000 Lux:
  Siang hari di bawah naungan awan.
- > 30.000 Lux:
  Terik matahari langsung (Outdoor).

STUDI KASUS 4

Menggabungkan Accelerometer dan Medan Magnetik untuk Menghitung Derajat Kompas

DigitalCompassActivity.java

```
private float[] gravityData = new float[3];
private float[] magneticData = new float[3];

@Override
public void onSensorChanged(SensorEvent event) {
    if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
        System.arraycopy(event.values, 0, gravityData, 0, 3);
    } else if (event.sensor.getType() == Sensor.TYPE_MAGNETIC_FIELD) {
        System.arraycopy(event.values, 0, magneticData, 0, 3);
    }

    float[] rMatrix = new float[9];
    float[] iMatrix = new float[9];
    // Hitung matriks rotasi dari fusi kedua sensor
    if (SensorManager.getRotationMatrix(rMatrix, iMatrix, gravityData, magneticData)) {
        float[] orientation = new float[3];
        SensorManager.getOrientation(rMatrix, orientation);

        // orientation[0] adalah sudut Azimuth dalam satuan Radian
        float azimuthInDegrees = (float) Math.toDegrees(orientation[0]);
        float compassAngle = (azimuthInDegrees + 360) % 360;
        ivCompassNeedle.setRotation(-compassAngle);
    }
}
```

PENJELASAN FUSI

1. getRotationMatrix():
Menghitung matriks transformasi rotasi koordinat bumi ke koordinat perangkat.

2. getOrientation():
Menghasilkan nilai Azimuth (Arah Utara), Pitch, dan Roll.

3. System.arraycopy():
Menghindari alokasi array baru demi performa tinggi.

BAGIAN VI // KEAMANAN & PRIVASI PENGGUNA

Izin (Permissions) & Privasi Pengguna di Java

Regulasi Privasi Android 10+, ACTIVITY_RECOGNITION, dan Sensor Kesehatan di Java

REGULASI PRIVASI

Android Mengatur Akses Sensor Berdasarkan Kebijakan Privasi Pengguna

Kategori Sensor	Jenis Izin (Permission)	Tingkat Izin	Mulai Diberlakukan
Motion & Environment (Accel, Light, Proximity)	Zero Permission (Tanpa Izin Runtime)	Normal	Semua Versi Android
Aktivitas Fisik (Step Counter, Step Detector)	ACTIVITY_RECOGNITION	Runtime Permission	Android 10 (API 29)+
Sensor Kesehatan (Heart Rate, Biometrik)	BODY_SENSORS	Runtime Permission	Android 6.0 (API 23)+
High Sampling Rate (> 200 Hz)	HIGH_SAMPLING_RATE_SENSORS	Normal (Manifest)	Android 12 (API 31)+

KEBIJAKAN HIGH SAMPLING RATE DI ANDROID 12+

Android 12 membatasi frekuensi sensor di atas 200 Hz guna menjaga stabilitas sistem dan efisiensi konsumsi daya komputasi perangkat. Pengembang wajib mendeklarasikan izin jika aplikasi memerlukan sampling berkecepatan tinggi.

ACTIVITY SENSORS

Membedakan Sensor Penghitung Langkah Akumulatif vs Detektor Langkah Real-Time di Java

StepActivity.java

```
// Menangani Izin Runtime ACTIVITY_RECOGNITION di Java
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
    if (ContextCompat.checkSelfPermission(this,
            Manifest.permission.ACTIVITY_RECOGNITION) != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.ACTIVITY_RECOGNITION}, 101);
    }
}

// Perbedaan Pembacaan Nilai di onSensorChanged():
if (event.sensor.getType() == Sensor.TYPE_STEP_COUNTER) {
    // event.values[0] = Total langkah akumulatif sejak perangkat reboot
    float totalSteps = event.values[0];
} else if (event.sensor.getType() == Sensor.TYPE_STEP_DETECTOR) {
    // event.values[0] selalu 1.0f setiap kali satu langkah kaki terjadi
    userStepCount++;
}
```

PERBANDINGAN KASUS

1. TYPE_STEP_COUNTER:
• Hemat daya (dihitung oleh chip sensor coprocessor).
• Ideal untuk aplikasi fitness pedometer harian yang mencatat total langkah.

2. TYPE_STEP_DETECTOR:
• Memicu event setiap langkah terayun.
• Ideal untuk animasi kaki karakter game atau analisis ritme joging secara langsung.

BAGIAN VII // PENGUJIAN, PRAKTIKUM & BEST PRACTICES

Pengujian Emulator & Best Practices Java

Simulasi Virtual Sensors di Android Studio, Spesifikasi Proyek Praktikum, dan 5 Prinsip Emas

TESTING & SIMULASI

Menguji Seluruh Sensor Tanpa Perangkat Fisik Menggunakan Panel Virtual Sensors Emulator

Langkah 1: Extended Controls

- Jalankan Android Emulator (Pixel API 34).
- Klik tombol titik tiga (...) pada bilah toolbar emulator di samping kanan.
- Jendela Extended Controls akan terbuka.

Langkah 2: Tab Virtual Sensors

- Pilih menu 'Virtual Sensors'.
- Gunakan sub-tab 'Device Pose' untuk memutar model 3D ponsel.
- Nilai Accelerometer dan Gyroscope akan otomatis terkirim ke aplikasi Java.

Langkah 3: Slider Lingkungan

- Buka sub-tab 'Additional Sensors'.
- Geser slider Light untuk mengubah nilai Lux (0 hingga 40.000 Lux).
- Ubah nilai Proximity dari 0 cm ke 5 cm untuk menguji fitur screen-off.

TUGAS MAHASISWA

Proyek Praktikum: Membangun Aplikasi Multi-Sensor Dashboard Berbasis Java Native

SPESIFIKASI PROYEK LAB

1. Nama Aplikasi: Multi-Sensor Dashboard Java.

2. Wajib Mengimplementasikan Minimal 3 Sensor:
   - 1 Sensor Gerak (Accelerometer / Gyroscope)
   - 1 Sensor Lingkungan (Light Sensor)
   - 1 Sensor Posisi (Proximity / Magnetic Field)

3. Fitur Interaktif Wajib:
   - Deteksi Shake untuk Reset Dashboard
   - Auto Dark Mode berdasarkan Lux sekitar
   - Indikator visual real-time

4. Pengumpulan:
   - Link Repositori GitHub Publik
   - Video Demo APK berdurasi 3 menit

RUBRIK PENILAIAN (TOTAL 100%)

- Bobot 30% - Fungsionalitas 3 Sensor:
Seluruh sensor terbaca dengan benar dan menampilkan data real-time.

- Bobot 25% - Higienitas Siklus Hidup (Lifecycle):
Registrasi tepat di onResume() dan unregisterListener() di onPause() tanpa exception.

- Bobot 25% - Logika Bisnis & Interaksi:
Algoritma shake detection dan transisi dark mode bekerja mulus dengan debounce.

- Bobot 20% - Desain UI & Penanganan Error:
Antarmuka rapi dengan CardView/RecyclerView dan memiliki pesan fallback jika sensor tidak ada.

RANGKUMAN KULIAH

Lima Prinsip Emas yang Wajib Diterapkan dalam Pemrograman Sensor Android Java

1. Always Check Availability

Selalu periksa accelerometer != null sebelum mendaftarkan listener untuk mencegah NullPointerException di Java.

2. Match Delay to Use Case

Gunakan delay rate terendah yang masih mencukupi kebutuhan UI (utamakan SENSOR_DELAY_UI).

3. Strict Lifecycle Hygiene

Wajib panggil unregisterListener(this) di dalam onPause() untuk mengamankan konsumsi baterai pengguna.

4. Zero Allocation in Callback

Jangan pernah membuat objek baru (new Object()) di dalam method onSensorChanged() demi kelancaran memori.

5. Graceful Hardware Fallback

Sediakan tampilan alternatif atau pesan informatif jika perangkat fisik pengguna tidak memiliki sensor yang diminta.